Branch Instructions

Prof. Gerardo Flores
February 2026
Microprocessor Systems (CSCE 3340 - SENG 3345)
TAMIU

Jump Instructions in ARM Assembly

- B (Branch): An unconditional jump to a label. Used for loops or to change execution flow.
- ~~BL (Branch with Link):~~ Calls a subroutine, storing the return address in the LR (Link Register).
- ~~BX LR (Branch and Exchange to Link Register):~~ Returns from a subroutine by jumping to the address stored in LR.

📌 Control Flow in ARM Assembly

Introduction to Control Flow

🔷 What is Control Flow?

- Control flow determines **how and when** instructions execute.
- In **assembly**, control flow is handled using **branches and conditionals**.

Unconditional Jump (B)

Assembly program
2-branch.s

🔷 B (Branch): Unconditional Jump

- Directly jumps to a **label** in the program.
- Used for **loops** and **changing execution flow**.

Example: Infinite Loop

```
.global _start

.section .text
_start:
    B loop      @ Jump to "loop"

loop:
    B loop      @ Infinite loop (keeps jumping to itself)
```

Comparisons and Conditionals (CMP, SUBS, BNE, BEQ)

Assembly program
2-if-else.s

🔷 Conditional Branching

- **CMP Rn, Rm** → Compares Rn with Rm and **updates flags**.
- **SUBS Rn, Rn, #1** → Subtracts and **updates flags**.
- BEQ **(Branch if Equal)** → Jumps if Z = 1 (values are equal).
- BNE **(Branch if Not Equal)** → Jumps if Z = 0 (values are different).

Example: If-Else Structure

```
.global _start

.section .text
_start:
    MOV R0, #5      @ Load 5 into R0
    MOV R1, #3      @ Load 3 into R1

    CMP R0, R1      @ Compare R0 and R1
    BEQ equal       @ If R0 == R1, jump to "equal"
    B not_equal     @ Otherwise, jump to "not_equal"

equal:
    MOV R2, #1      @ Set R2 = 1 if equal
    B end

not_equal:
    MOV R2, #0      @ Set R2 = 0 if not equal

end:
    B end           @ Infinite loop (end program)
```

In **ARM assembly**, the **If-Else structure** follows this logic:

```
CMP Rn, Rm        @ Compare two registers (Rn - Rm)
BEQ if_block      @ If they are equal, jump to if_block
B else_block      @ Otherwise, jump to else_block

if_block:
    @ Code for the "if" case
    B end_if

else_block:
    @ Code for the "else" case

end_if:
```

•original

```
.global _start

.section .text
_start:
    MOV R0, #5      @ Load 5 into R0
    MOV R1, #3      @ Load 3 into R1

    CMP R0, R1      @ Compare R0 and R1
    BEQ equal       @ If R0 == R1, jump to "equal"
    B not_equal     @ Otherwise, jump to "not_equal"

equal:
    MOV R2, #1      @ Set R2 = 1 if equal
    B end

not_equal:
    MOV R2, #0      @ Set R2 = 0 if not equal

end:
    B end           @ Infinite loop (end program)
```

```
.global _start

.section .text
_start:
    MOV R0, #7      @ Load 5 into R0
    MOV R1, #7      @ Load 3 into R1

    CMP R0, R1      @ Compare R0 and R1

    BNE not_equal       @ If R0 == R1, jump to "equal"
    B equal     @ Otherwise, jump to "not_equal"

equal:
    MOV R2, #1      @ Set R2 = 1 if equal
    B end

not_equal:
    MOV R2, #0      @ Set R2 = 0 if not equal

end:
    B end           @ Infinite loop (end program)
```

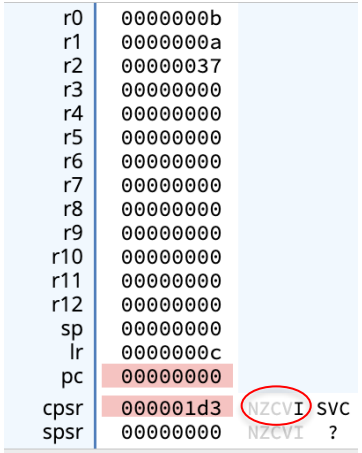
Update Flags in ARM Assembly (CPSR - Current Program Status Register)

When executing instructions like CMP and SUBS, these **update the flags in the processor's status register called:**

(CPSR - Current Program Status Register).

These flags are crucial for executing **conditional branches** (BEQ, BNE, etc.).

CPSR



- You will see the **important flags**:
- N (Negative)
- Z (Zero)
- C (Carry)
- V (Overflow)

🔍 These flags change automatically after certain instructions like CMP, SUBS, and arithmetic operations.

Note:

r0	0000000b	
r1	0000000a	
r2	00000037	
r3	00000000	
r4	00000000	
r5	00000000	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	0000000c	
pc	00000000	
cpsr	000001d3	NZCVI SVC
spsr	00000000	NZCVI ?

The I flag is the **Interrupt Disable Flag**. When set (I = 1), it **disables IRQ (Interrupt Requests)**, meaning the processor will ignore normal hardware interrupts.

📌 **What is SVC Mode? (Supervisor Mode)**

- SVC stands for **Supervisor Mode**, which is a **privileged execution mode** in ARM processors.
- The processor enters **SVC mode** when:
  - It starts executing after reset.
  - A software interrupt (SVC instruction) is triggered.

Important Flags and How They Work

Flag	Meaning	How It Is Activated
Z (Zero Flag)	Indicates if the result is `0`.	Activated if `CMP` or `SUBS` results in `0`.
N (Negative Flag)	Indicates if the result is negative.	Activated if the result has the most significant bit as `1`.
C (Carry Flag)	Indicates if there was a carry in an operation.	Activated if there is a carry in an addition or subtraction.
V (Overflow Flag)	Indicates if there was arithmetic overflow.	Activated if the result exceeds the representable range.

Example 1: How CMP Updates the Z Flag

Assembly program
2-Z-flag.s

```
.global _start

.section .text
_start:
    MOV R0, #5      @ Load 5 into R0
    MOV R1, #5      @ Load 5 into R1

    CMP R0, R1      @ Compare R0 and R1 (5 - 5 = 0)
    BEQ equal       @ If result is 0, jump to "equal"

    B end_program

equal:
    MOV R2, #1      @ If R0 == R1, store 1 in R2

end_program:
    B end_program   @ Infinite loop
```

✅ When CMP R0, R1 detects 5 - 5 = 0, the Z flag is set to 1, and BEQ equal is executed.

r0	00000005	
r1	00000005	
r2	00000001	
r3	00000000	
r4	00000000	
r5	00000000	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	00000018	
cpsr	600001d3	NZCVI SVC
spsr	00000000	NZCVI ?

🔍 Run this code in CPulator and check the CPSR to see how the Z flag changes.

Example 2: How SUBS Adjusts Flags

Assembly program
2-SUBS-Adjusts-Flags.s

✅ Here, SUBS updates the Z flag when R0 reaches 0, stopping the loop.

```
.global _start

.section .text
_start:
    MOV R0, #3      @ Load 3 into R0

loop:
    SUBS R0, R0, #1  @ R0 = R0 - 1 and updates flags
    BNE loop         @ If R0 ≠ 0, repeat loop

end:
    B end            @ Stop execution
```

r0	00000000	
r1	00000000	
r2	00000000	
r3	00000000	
r4	00000000	
r5	00000000	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	0000000c	
cpsr	600001d3	NZCVI SVC
spsr	00000000	NZCVI ?

✅ Check how the Z flag changes in CPulator on each iteration.

Difference Between SUBS and SUB in ARM Assembly

In ARM Assembly, both SUB and SUBS perform **subtraction**, but the key difference is whether or not the **flags in the CPSR (Current Program Status Register)** are updated.

📌 `SUB` - Subtract Without Updating Flags

- Performs a subtraction but **does not modify** the condition flags (`N`, `Z`, `C`, `V`).
- Useful when you only need the result and don't need to check conditions.

📌 `SUBS` - Subtract and Update Flags

- Performs a subtraction **and updates** the condition flags (`N`, `Z`, `C`, `V`).
- Used in **loops and condition checks** where we need to know if the result is zero, negative, or causes an overflow.

Instruction	Operation	Updates Flags?	Use Case
`SUB`	`Rd = Rn - Rm`	❌ No	Simple subtraction when flags are not needed
`SUBS`	`Rd = Rn - Rm`	✅ Yes	Subtraction with condition checking (loops, comparisons)

While Loop Using CMP and BNE

Assembly program
2-while.s

◆ while Loop in Assembly

- Uses CMP to compare values.
- Uses BNE to **repeat while condition is true**.

Example: Counting Down from 5 to 0

```c
c

#include <stdio.h>

int main() {
    int R0 = 5;  // Load counter (R0 = 5)

    while (R0 > 0) {  // Loop until R0 == 0
        R0--;  // Decrement counter
    }

    // Infinite loop (similar to "B end_loop" in assembly)
    while (1) {
        // Halt execution
    }

    return 0;
}
```

```
.global _start

.section .text
_start:
    MOV R0, #5      @ Load counter (R0 = 5)

loop:
    CMP R0, #0      @ Compare R0 with 0
    BEQ end_loop    @ If R0 == 0, exit loop

    SUBS R0, R0, #1  @ Decrement counter
    B loop          @ Repeat loop

end_loop:
    B end_loop      @ Stop execution
```

r0	00000000	
r1	00000003	
r2	00000008	
r3	00000000	
r4	00000000	
r5	00000000	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	0000000c	
pc	00000004	
cpsr	600001d3	NZCVI SVC
spsr	00000000	NZCVI ?

For Loop in Assembly

Assembly program
2-summing-1-10.s

for **Loop in Assembly**

- Similar to while, but structured explicitly with:
  - **Initialization**
  - **Condition Check (CMP)**
  - **Decrement and Repeat (BNE)**

Example: Summing Numbers from 1 to 10

```
.global _start

.section .text
_start:
    MOV R0, #1      @ Initialize counter (i = 1)
    MOV R1, #10     @ Set limit (10)
    MOV R2, #0      @ Initialize sum (sum = 0)

loop:
    CMP R0, R1      @ Compare i with 10
    BGT end_loop    @ If i > 10, exit loop

    ADD R2, R2, R0  @ sum = sum + i
    ADD R0, R0, #1  @ i = i + 1
    B loop          @ Repeat loop

end_loop:
    B end_loop      @ Stop execution
```

```c
#include <stdio.h>

int main() {
    int sum = 0;  // Initialize sum

    // Loop from 1 to 10
    for (int i = 1; i <= 10; i++) {
        sum += i;  // sum = sum + i;
    }

    // Infinite loop to simulate end_loop (not necessary in real C programs)
    while (1);

    return 0;
}
```

Classroom Exercises

Exercise 1: While Loop

CMP and BNE, BGT, …

- Write an assembly program to **sum numbers from 1 to 10** using a while loop.

Exercise 2: If-Else Condition

- Write a program that:
  - Compares two numbers.
  - If they are **equal**, store 1 in R2.
  - If they are **not equal**, store 0 in R2.

In **ARM assembly**, the **If-Else structure** follows this logic:

```
CMP Rn, Rm        @ Compare two registers (Rn - Rm)
BEQ if_block      @ If they are equal, jump to if_block
B else_block      @ Otherwise, jump to else_block

if_block:
    @ Code for the "if" case
    B end_if

else_block:
    @ Code for the "else" case

end_if:
```

🔍 Try these on CPulator and discuss the results!

Exercise 1

Assembly program
2-ex1.s

Write an assembly program to **sum numbers from 1 to 10** using a while loop.

```
.global _start

.section .text
_start:
    MOV R0, #1        @ Initialize counter (i = 1)
    MOV R1, #10       @ Set limit (10)
    MOV R2, #0        @ Initialize sum (sum = 0)

loop:
    CMP R0, R1        @ Compare i with 10
    BGT end_loop      @ If i > 10, exit loop

    ADD R2, R2, R0    @ sum = sum + i
    ADD R0, R0, #1    @ i = i + 1
    B loop            @ Repeat loop

end_loop:
    B end_loop        @ Stop execution (infinite loop)
```

Exercise 2

Assembly program
2-ex2.s

Write a program that compares two numbers.

- If they are **equal**, store 1 in R2.
- If they are **not equal**, store 0 in R2.

```
.global _start

.section .text
_start:
    MOV R0, #5      @ Load first number (R0 = 5)
    MOV R1, #5      @ Load second number (R1 = 5)

    CMP R0, R1      @ Compare R0 and R1
    BEQ equal       @ If R0 == R1, jump to "equal"

    MOV R2, #0      @ Else, store 0 in R2
    B end_program   @ Skip "equal" section

equal:
    MOV R2, #1      @ Store 1 in R2 if equal

end_program:
    B end_program   @ Infinite loop (stop execution)
```

Summary

- **B** → Unconditional jump.
- **CMP** → Compares two values.
- **BEQ / BNE** → Conditional jumps.
- **while loop** → Uses CMP and BNE.
- **for loop** → Uses CMP, BNE, and a counter.

🔍 Next Steps

- Experiment with different values in the simulator.
- Try nested loops and conditions.
- Next class: **<u>Functions</u> and <u>Subroutines</u> in Assembly (BL and BX LR)!**

Jump Instructions in ARM Assembly

- B (Branch): An unconditional jump to a label. Used for loops or to change execution flow.
- BL (Branch with Link): Calls a subroutine, storing the return address in the LR (Link Register).
- ~~BX LR (Branch and Exchange to Link Register):~~ Returns from a subroutine by jumping to the address stored in LR.

Motivation example

C Program	Assembly Program

`int t;`
`int sum2(int a1, int a2);`
	`AREA sum, CODE`
	`EXPORT __main`
	`ALIGN`
	`ENTRY`

`int main(){`	`__main PROC`
	`MOV r0, #1 ; 1st argument`
	`MOV r1, #2 ; 2nd argument`

`t = sum2(1, 2);`	`BL sum2 ; result returned in r0`
	`; LR points to LDR instr`
	`LDR r1, =t ; memory address of t`
	`STR r0, [r1] ; save the sum`

`while(1);` `}`	`stop B stop` `ENDP`
`int sum2(int a1, int a2){`	`sum2 PROC ; name of procedure` `; r0 = 1st argument`
`int total;`	`; r1 = 2nd argument`
`total = a1 + a2;`	`ADD r0, r0, r1 ; r0 = r0 + r1` `; r0 = result to be returned`
`return total;` `}`	`BX lr ; set PC to LR` `ENDP ; end of procedure`

	`AREA myData, DATA`
	`t DCD 0`
	`END ; end of program`

Example 8-5. Passing two arguments to a subroutine

Motivation example

```
1      .data
2      t:  .word  0          @ Memory location to store the sum
3
4      .text
5      .global _start
6
7  _start:
8      MOV R0, #1          @ Load first argument (1) into R0
9      MOV R1, #2          @ Load second argument (2) into R1
10
11     BL sum2             @ Call sum2 subroutine (result returned in R0)
12
13     LDR R1, =t          @ Load the address of variable t into R1
14     STR R0, [R1]        @ Store the result of sum in memory at address t
15
16 stop:
17     B stop              @ Infinite loop to stop execution
18
19 sum2:
20     @ Subroutine to add two numbers
21     @ Input:  R0 (first argument), R1 (second argument)
22     @ Output: R0 (sum of R0 and R1)
23
24     ADD R0, R0, R1      @ Perform addition R0 = R0 + R1
25     BX LR               @ Return to caller using LR (Link Register)
26
27     .end
```

LR

Assembly program
2-motiv.s

r0	00000003	
r1	00000028	
r2	0000000a	
r3	00000006	
r4	00000000	
r5	00000000	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	0000000c	
pc	00000014	
cpsr	000001d3	NZCVI SVC
spsr	00000000	NZCVI ?

Functions and Subroutines in ARM Assembly

Objectives

By the end of this lesson, students will:

- Understand how functions and subroutines work in ARM Assembly.
- Learn how to use **BL (Branch with Link)** and **BX LR (Branch and Exchange)** for function calls.
- Understand the role of the **LR (Link Register)**.
- Practice writing and calling subroutines in ARM Assembly.

Why Use Functions in Assembly?

Functions (or subroutines) allow us to:

- **Modularize** the code: Avoid repetition.
- **Improve readability**: Code is easier to understand.
- **Reuse logic**: Write once, use multiple times.
- **Optimize memory usage**: Instead of duplicating instructions, we call a function.

Before we start…

Assembly program
2-motiv.s

Observe the behavior of PC and LR in each line

r0	00000001
r1	00000000
r2	00000000
r3	00000000
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	00000004
cpsr	000001d3
spsr	00000000

```
                                  4   .text
                                  5   .global _start
                                  7   _start:
                                  8   MOV R0, #1        @ Load first argument (1) into R0
                                  _start:
00000000  e3a00001                    mov     r0, #1  ; 0x1
                                  9   MOV R1, #2        @ Load second argument (2) into R1
00000004  e3a01002                    mov     r1, #2  ; 0x2
                                  11  BL sum2           @ Call sum2 subroutine (result returned in R0)
00000008  eb000002                    bl  0x18  (0x18: sum2)
                                  13  LDR R1, =t        @ Load the address of variable t into R1
0000000c  e59f100c                    ldr     r1, [pc, #12]   ; 0x20
                                  14  STR R0, [R1]      @ Store the result of sum in memory at address t
00000010  e5810000                    str     r0, [r1]
                                  16  stop:
                                  17  B stop            @ Infinite loop to stop execution
                                  stop:
00000014  eafffffe                    b   0x14  (0x14: stop)
                                  19  sum2:
                                  20  @ Subroutine to add two numbers
                                  21  @ Input:  R0 (first argument), R1 (second argument)
                                  22  @ Output: R0 (sum of R0 and R1)
                                  24  ADD R0, R0, R1    @ Perform addition R0 = R0 + R1
                                  sum2:
00000018  e0800001                    add     r0, r0, r1
                                  25  BX LR             @ Return to caller using LR (Link Register)
0000001c  e12fff1e                    bx  lr
                                  13  LDR R1, =t        @ Load the address of variable t into R1
00000020  00000028                    andeq   r0, r0, r8, LSR #32
00000024  00000000                    andeq   r0, r0, r0
                                  t:
00000028  00000000                    andeq   r0, r0, r0
0000002c  00000000                    andeq   r0, r0, r0
                                  _end:
```

Yellow line points to the next instruction.

The Link Register (LR) and Function Calls

BL add_numbers overwrites **LR** with the return address for add_numbers .

How Function Calls Work

In ARM Cortex-M, when calling a function:

1. The **BL (Branch with Link)** instruction stores the return address in the **LR (Link Register)**.
2. The CPU jumps to the function's memory address.
3. At the end of the function, the **BX LR** instruction returns execution to the saved address in **LR**.

Example of Function Call in ARM Assembly

```
.global _start

.section .text
_start:
    MOV R0, #5        // Load first number into R0
    MOV R1, #3        // Load second number into R1
    BL add_numbers    // Call function (saves return address in LR)
    B _start          // Infinite loop after function call

add_numbers:
    ADD R0, R0, R1    // Perform addition: R0 = R0 + R1
    BX LR             // Return to the caller (uses value in LR)
```

Assembly program
2-call.s

Explanation

1. BL add_numbers:
    1. Saves the return address in **LR**.
    2. Jumps to add_numbers.
2. ADD R0, R0, R1:
    1. Adds the two numbers.
3. BX LR:
    1. Uses **LR** to return to the instruction after BL.

Example of Function Call in ARM Assembly

```
.global _start

.section .text
_start:
    MOV R0, #5        // Load first number into R0
    MOV R1, #3        // Load second number into R1
    BL add_numbers    // Call function (saves return address in LR)
    B _start          // Infinite loop after function call

add_numbers:
    ADD R0, R0, R1    // Perform addition: R0 = R0 + R1
    BX LR             // Return to the caller (uses value in LR)
```

Start

r0	00000000
r1	00000000
r2	00000000
r3	00000000
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	00000000
cpsr	000001d3
spsr	00000000

MOV R0, #5

r0	00000005
r1	00000000
r2	00000000
r3	00000000
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	00000004
cpsr	000001d3
spsr	00000000

MOV R1, #3

r0	00000005
r1	00000003
r2	00000000
r3	00000000
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	00000008
cpsr	000001d3
spsr	00000000

BL add_numbers

r0	00000005
r1	00000003
r2	00000000
r3	00000000
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	0000000c
pc	00000010
cpsr	000001d3
spsr	00000000

ADD R0, R0, R1

r0	00000008
r1	00000003
r2	00000000
r3	00000000
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	0000000c
pc	00000014
cpsr	000001d3
spsr	00000000

BX LR

r0	00000008
r1	00000003
r2	00000000
r3	00000000
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	0000000c
pc	0000000c
cpsr	000001d3
spsr	00000000

B _start

r0	00000008
r1	00000003
r2	00000000
r3	00000000
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	0000000c
pc	00000000
cpsr	000001d3
spsr	00000000

MOV R0, #5

r0	00000005
r1	00000003
r2	00000000
r3	00000000
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	0000000c
pc	00000004
cpsr	000001d3
spsr	00000000

. . .

Exercises for Students

1. **Write a function** that multiplies two numbers and returns the result.
2. **Modify the factorial function** to compute factorial iteratively instead of recursively.
3. **Write a function** that swaps the values of **R0** and **R1**.

1. Write a function that multiplies two numbers and returns the result.

Assembly program
2-multiplication.s

2. Compute the factorial of a given number iteratively instead of **recursively**.

Assembly program
2-factorial_1.s

An **iterative** approach uses loops (such as for, while, or do-while) to **repeat** a set of instructions until a condition is met.

A **recursive** approach is when a function **calls itself** until it reaches a base case.

```
factorial:
    CMP R0, #1        @ If R0 == 1, return 1
    BEQ base_case
    PUSH {LR}         @ Save return address
    SUB R0, R0, #1    @ n - 1
    BL factorial      @ Recursive call
    POP {LR}          @ Restore return address
    MUL R0, R0, R1    @ Multiply n * factorial(n-1)
    BX LR             @ Return
```

3. Write a function that swaps the values of **R0** and **R1**.

Assembly program
2-swap.s

Saving LR on the Stack

Assembly program
2-err.s

If a function calls another function, **LR gets overwritten**. To prevent this, we **save it on the stack** using **PUSH/POP**.

Prediction exercise: Analyze the behavior of the nested function call. Execute the program step by step. Identify and explain what goes wrong in the code.

```
.global _start

.section .text
_start:
    MOV R0, #5              @ Load test value into R0
    BL first_function       @ Call first function
    B end_program           @ End program

first_function:
    MOV R1, #10             @ Do something (dummy operation)
    BL second_function      @ Call another function (overwrites LR)
    @ At this point, LR is overwritten and cannot return correctly
    BX LR                   @ Return, but LR is now set by second_function

second_function:
    MOV R2, #20             @ Another dummy operation
    BX LR                   @ Return to first_function (but LR in first_function is lost)

end_program:
    B end_program           @ Infinite loop to stop the program
```

The Concept of Stack and Its Usage in Microprocessors

In general we can describe the ***stack*** as a temporary storage for data. The access to data in the stack is organized as **Last In First Out (LIFO)**, which means that the last data stored in the stack is the first that can be retrieved, and the first data stored in the stack is the last to be retrieved.

- The operation of adding data to the stack is called **PUSH** and
- the operation of retrieving data is called **POP(PULL)**.

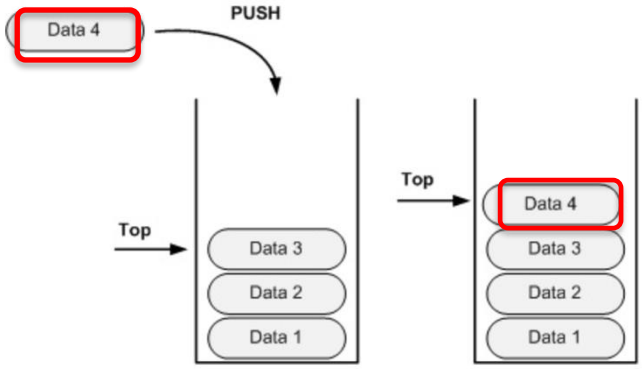


Fig. 1 Push Operation

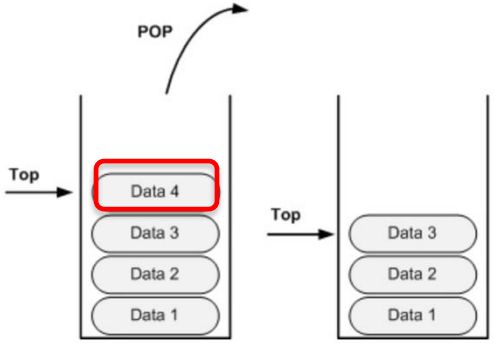


Fig. 2 Pop Operation

The stack can be viewed as a pile of plates. The bottom plate is the first data pushed onto the stack and the top plate is the last data pushed. When the top plate is removed (pulled), the one below pops up to become the new top. The top plate is pulled first and the bottom plate is pulled last. It is important to note that only the top element in the stack can be accessed (in the classic stack implementation).

The stack is a section of memory in RAM used for temporarily storing data such as register values, return addresses, and local variables. Its location and size depend on the **Stack Pointer (SP)**, which is a special register that **points to the top of the stack**.

r0	00000006	
r1	0000000a	
r2	0000000a	
r3	00000006	
r4	00000000	
r5	00000000	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	ed5555b8	
cpsr	000001d3	NZCVI SVC
spsr	00000000	NZCVI ?

Where is the Stack Located?

Stack Pointer (SP Register):

1. The **SP (Stack Pointer) register** holds the current address of the top of the stack.
2. Every time you PUSH, the SP **decreases** (stack grows down).
3. Every time you POP, the SP **increases** (stack shrinks up).

Saving LR on the Stack

Assembly program
2-nested.s

Example: Nested Function [solution to the previous example]

```
.global _start

.section .text
_start:
    MOV R0, #7
    MOV R1, #4
    BL first_function
    B _start

first_function:
    PUSH {LR}          // Save LR before calling another function
    BL second_function
    POP {LR}           // Restore LR before returning
    BX LR

second_function:
    ADD R0, R0, R1     // Add R0 and R1
    BX LR
```

Explanation

1. PUSH {LR} saves the return address before calling another function.

2. BL second_function overwrites LR.

3. POP {LR} restores the original return address before BX LR.

Example: Nested Function

```
.global _start

.section .text
_start:
    MOV R0, #7
    MOV R1, #4
    BL first_function
    B _start

first_function:
    PUSH {LR}          // Save LR before calling another function
    BL second_function
    POP {LR}           // Restore LR before returning
    BX LR

second_function:
    ADD R0, R0, R1     // Add R0 and R1
    BX LR
```

sp	fffffffc
lr	0000000c
pc	00000014

Assembly program
2-nested.s

Go to address, label, or register: fffffffc Refresh

Address	Memory contents and ASCII				
ffffff70	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	....
ffffff80	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	....
ffffff90	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	....
ffffffa0	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	....
ffffffb0	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	....
ffffffc0	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	....
ffffffd0	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	....
ffffffe0	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	....
fffffff0	aaaaaaaa	aaaaaaaa	aaaaaaaa	0000000c	....
00000000	e3a00007	e3a01004	eb000000	eafffffb	....
00000010	e52de004	eb000001	e49de004	e12fff1e	..-. ..../.

sp	fffffffc
lr	00000018
pc	00000020

How POP {LR} Works

- The **top value** from the stack (pointed to by SP) is **loaded into LR**.
- Then, SP is **incremented** to remove that value from the stack.

PUSH and POP always operate on the stack. Whatever is inside {} **is stored in or retrieved from the stack memory**.

REMEMBER:

2. Every time you PUSH, the SP **decreases** (stack grows down).

Question: What is the result of this line assuming LR = 8:

MOV R0, #7
PUSH {LR, R0}

ffffffc0	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa
ffffffd0	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa
ffffffe0	aaaaaaaa	aaaaaaaa	00000005	00000020
fffffff0	00000006	00000020	00000007	00000008

Investigate the result of these lines:

POP {LR, R0}

POP {R0, LR}

Exercise: Using the Stack with PUSH and POP

Write an assembly program that performs the following steps:

1. Load the value 0x10 into register R0 and the value 0x20 into register R1.
2. Use **PUSH** to store both values on the stack.
3. Clear registers R0 and R1 (set them to zero).
4. Use **POP** to retrieve the values from the stack back into R0 and R1.

At the end of the program:

- R0 should hold 0x10
- R1 should hold 0x20

Example: Using Functions to Compute Factorial

The code may appear simple at first glance, but it is more intricate than it seems. Study it carefully, analyze each line, and provide a thorough explanation of its functionality and underlying logic.

Let's write a function that calculates **factorial(n) = n × (n-1) × ... × 1** using recursion.

```
.global _start

.section .text
_start:
    MOV R0, #4          @ n = 4 (input for factorial)
    BL factorial        @ Call factorial function
    B end_program       @ Infinite loop to stop

factorial:
    CMP R0, #1          @ Compare R0 with 1 (if n == 1, return 1)
    BEQ base_case       @ If R0 == 1, jump to base_case

    PUSH {LR, R0}       @ Save LR and n (R0) in stack
    SUB R0, R0, #1      @ n - 1
    BL factorial        @ Recursive call

    POP {R1, LR}        @ Restore n (R1) and LR (return address)
    MUL R0, R0, R1      @ R0 = factorial(n-1) * n
    BX LR               @ Return

base_case:
    MOV R0, #1          @ Base case: factorial(1) = 1
    BX LR               @ Return

end_program:
    B end_program       @ Stop execution
```

- **PUSH {LR, R0}** → Saves the return address (LR) and n (R0) onto the stack before making the recursive call.
- **POP {R1, LR}** → Restores n into R1 and the return address into LR from the stack.
- **MUL R0, R0, R1** → Multiplies the result of the recursion (R0) by n (R1).
- **BX LR** → Jumps back to the return address stored in LR.

The **stack grows and shrinks properly** with each call, because **every PUSH has a corresponding POP**.

Key Differences Between BL/BX and PUSH/POP

PUSH and POP are used to save and restore registers, but they do **not replace BL and BX** when calling functions.

Instruction	Purpose
`BL function`	Calls a function and saves the **return address in** `LR`
`BX LR`	Returns from the function using the address stored in `LR`
`PUSH {LR}`	Saves `LR` (return address) on the stack (useful for nested function calls)
`POP {LR}`	Restores `LR` before returning (used when `LR` was pushed)

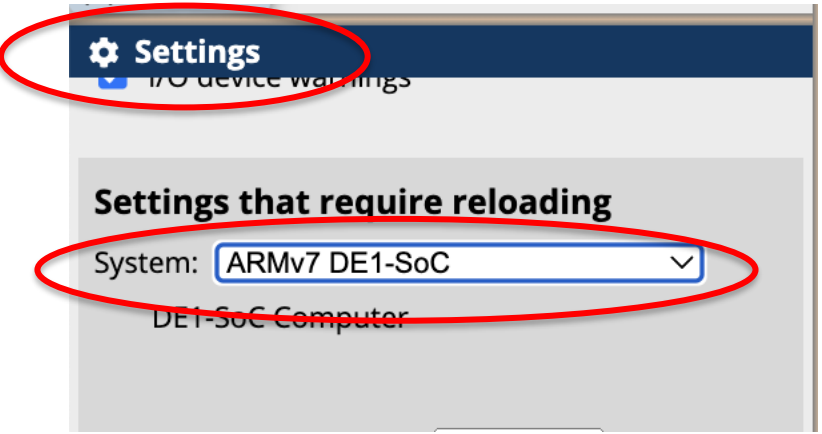
Why Use PUSH and POP With Functions?

When calling **nested functions (functions calling functions)**, LR gets **overwritten**. To avoid losing the return address, we **store it using PUSH** before calling another function and **restore it using POP** before returning.

Summary

- **BL (Branch with Link)** saves the return address in **LR**.
- **BX LR** returns to the saved address.
- **PUSH {LR} / POP {LR}** is used when calling multiple functions to prevent overwriting **LR**.
- Functions make assembly code **modular**, **reusable**, and **readable**.

For the next exercises, choose ARMv7 DE1-SoC in your cpulator and the devices will appear



Exercise 1 – Turn On LEDs with a Fixed Value

Problem Description:

Write an ARM assembly program that **turns on the first four LEDs** on the CPulator simulation by writing a **fixed value** to the LED device register.

- **LEDs Memory Address:** 0xFF200000
- **Fixed Value to Display:** 0x0F (0000 1111 in binary – turns on the first 4 LEDs)

Objective:

- Practice **basic memory-mapped I/O**.
- Learn to **write values to GPIO devices** (LEDs).
- **No need for PUSH/POP** because we **don't call subroutines in this simple version**.
- **Stop the program after turning on the LEDs**.

General Strategy for Solving Assembly Code Problems

1. Understand the Problem (Define the Goal)

Before coding, clearly understand the problem by [read the problem as many times as needed]:

- **Identifying the task** (e.g., read a sensor, turn on an LED, perform a computation).
- **Determining the conditions** under which each action is executed.
- **Identify key memory addresses** for peripherals or registers.

✅ **Example:**
Read a button state and turn on an LED when pressed.

General Strategy for Solving Assembly Code Problems

2. Identify the Need for a Loop

A loop is necessary when:

- Constant monitoring of an input is required.
- Repetitive actions need to be executed (e.g., value updates, periodic tasks).

✅ **Example:**
The button state changes dynamically, requiring continuous checking.

General Strategy for Solving Assembly Code Problems

3. Break the Problem into Blocks

Divide the problem into smaller tasks with specific actions.

Task	Action Needed
Read input	Load value from memory (LDR)
Compare value	Check against a constant (CMP)
Execute action	Branch to correct function (BEQ, BNE)
Turn on LED	Write '1' to the LED memory (STR)
Turn off LED	Write '0' to the LED memory (STR)
Repeat	Loop back to the start (B loop)

monitoring

function 1
function 2

loop

✅ **Example:**
If the button is pressed [monitoring], turn on the LED [function 1]; otherwise, turn it off [function 2].

General Strategy for Solving Assembly Code Problems

4. Plan the Code Structure

Define the logical execution flow.

```
[Initialization] → Set up addresses

Loop:
    [Input] → Read button state
    [Processing] → Compare with ‘1’
        YES → [Turn LED ON]
        NO → [Turn LED OFF]
    [Repeat] → Go back to Input
```

✅ **Example:**
If the button is pressed, write ‘1’ to the LED; otherwise, write ‘0’.

General Strategy for Solving Assembly Code Problems

5. Test and Debug

- **Simulation:** Run the code on tools like Cpulator, or any ARM-supported simulator.
- **Debugging:** Inspect registers and memory values to verify correct execution.
- **Optimization:** Remove unnecessary instructions for efficiency.

Summary of the Method

✅ **1. Understand the problem.**
✅ **2. Determine if a loop is needed.**
✅ **3. Break it down into blocks.**
✅ **4. Plan the code structure.**
✅ **5. Write and optimize assembly code.**
✅ **6. Test and debug in a simulator or real hardware.**

Exercise 2 – Toggle LEDs Based on Switch Input

Problem Description:

Write an ARM assembly program that **reads the state of the first switch** and **turns on the first LED if the switch is ON**. If the switch is **OFF**, the LED should be turned **OFF**.

- **LEDs Memory Address:** 0xFF200000
- **Switches Memory Address:** 0xFF200040

Objective:

- Practice **basic input and output operations** using **GPIO devices**.
- **Introduce conditional branches** (CBZ, B, CMP, etc.).
- **Introduce simple loop** to **continuously check input and update output**.
- **No PUSH/POP needed yet**, but the loop **emphasizes flow control**.

🧠 How to Approach the Problem – General Plan

1. Understand the Problem (Goal)

- **Read the switch state.**
- **Turn the LED ON if the switch is ON (1).**
- **Turn the LED OFF if the switch is OFF (0).**
- **Keep checking continuously** (loop).

2. Identify the Need for a Loop

Visual flow:

We need to **check the switch over and over**, so **<u>we need a loop</u>**.

```
[Start] → [Read Switch]
                |
                v
     Is Switch ON?
        /       \
    YES          NO
   /               \
[LED ON]        [LED OFF]
    \               /
     ----> [Repeat]
```

3. Break It Down Into Blocks

Task	Action Needed
Read Switch	**Read memory (input)**
Compare Value	**Compare with 1 (ON)**
Turn LED ON	**Write 1 to LED (output)**
Turn LED OFF	**Write 0 to LED (output)**
Repeat	**Go back to start (loop)**

4. Plan the Code Structure (Big Picture)

```
[Initialization] → Set up addresses

Loop:
    [Input] → Read switch state
    [Processing/Decision] → Is switch ON?
        YES → [Output] Turn LED ON      ──→ functionOn
        NO  → [Output] Turn LED OFF     ──→ functionOff
    [Repeat] → Loop back to Input
```

```
[Set Memory Addresses]
        ↓
[Loop Start]
        ↓
[Read Switch]
        ↓
[Compare with 1]
    /         \
 YES           NO
 /               \
[LED ON]        [LED OFF]
    \               /
     -----> [Loop Start]
```

Exercise 3 – Toggle LEDs Continuously Using a Subroutine (BL, PUSH/POP)

Problem Description:

Write an ARM assembly program that **calls a subroutine to toggle the LEDs continuously**. Each time the subroutine is called, **the state of LEDs should change (ON → OFF or OFF → ON)**. Introduce a **delay between toggles** using a simple loop.

This exercise will **focus on subroutines (BL), stack operations (PUSH/POP)**, and **simple delays using loops**.

Objectives:

- Practice **subroutines with BL** (Branch with Link).
- Use **PUSH/POP** to **save and restore registers**.
- Create **a simple delay loop**.
- Control **LEDs output** continuously.

EOR R1, R1, #0xFF

XOR

$0 \oplus 0 = 0$

$0 \oplus 1 = 1$

$1 \oplus 0 = 1$

$1 \oplus 1 = 0$

The Divide-and-Conquer Approach

To efficiently solve this problem, we break it down into three essential components:

1. **Delay Function:** Introduces a simple delay using a counter loop.
2. **LED Toggle Function:** Uses a **subroutine** to toggle the LED state by flipping bits in a register.
3. **Main Program:** Organizes and coordinates the previous functions.

By dividing the problem into these manageable tasks, we ensure modularity and reusability.

In this approach, we need to know how to solve short and simple programs. Gaining experience in solving such programs is highly useful.

Delay function [part of problem 3]

Assembly program
2-delay.s

```
.global _start

.equ DELAY_COUNT, 0x00FFFFFF  @ Define delay count

_start:
    BL delay       @ Call delay function
    B _start       @ Repeat indefinitely

delay:
    PUSH {LR}        @ Save return address
    LDR R2, =DELAY_COUNT  @ Load delay counter
delay_loop:
    SUBS R2, R2, #1  @ Decrement counter
    BNE delay_loop   @ Repeat until counter is zero
    POP {LR}         @ Restore return address
    BX LR            @ Return from subroutine
```

Toggle function [part of problem 3]

Assembly program
2-toggle.s

```
.global _start

.equ LEDS, 0xFF200000   @ LED memory address

_start:
    LDR R0, =LEDS   @ Load LED address
    MOV R1, #0      @ Initialize LED state (OFF)

loop:
    BL toggle_leds  @ Call subroutine to toggle LEDs
    B loop          @ Repeat loop indefinitely

toggle_leds:
    LDR R1, [R0]        @ Read current LED state
    EOR R1, R1, #0xFF   @ Toggle LED state (flip bits)
    STR R1, [R0]        @ Store new state to LEDs
    BX LR               @ Return from subroutine
```

Solution to Exercise 3

Assembly program
2-ex3.s

```
.global _start

.equ LEDS, 0xFF200000      @ Address for LEDs

_start:
    LDR R0, =LEDS          @ Load LED base address into R0
    MOV R1, #0             @ Initialize LED state (OFF)

loop:
    BL toggle_leds         @ Call subroutine to toggle LEDs
    BL delay               @ Call subroutine for delay
    B loop                 @ Repeat loop indefinitely

toggle_leds:
    @PUSH {LR}              @ Save return address

    EOR R1, R1, #0xFF      @ Toggle R1 (flip bits, 0 → FF, FF → 0)
    STR R1, [R0]           @ Write new state to LEDs

    @POP {LR}               @ Restore return address
    BX LR                  @ Return from subroutine

delay:
    @PUSH {LR}              @ Save return address

    LDR R2, =0x00FFFFF     @ Load delay counter
delay_loop:
    SUBS R2, R2, #1        @ Decrement counter
    BNE delay_loop         @ Repeat until counter is 0

    @POP {LR}               @ Restore return address
    BX LR                  @ Return from subroutine
```

1. **Delay Function:** Introduces a simple delay using a counter loop.
2. **LED Toggle Function:** Uses a **subroutine** to toggle the LED state by flipping bits in a register.
3. **Main Program:** Organizes and coordinates the previous functions.

Reminder:

For the next class, please submit the list of teams for the final project.

- There is no maximum number of members per team, but each team must have at least 6 members due to the high number of students in the course.

Exercise I - Turn on a single LED

Problem Description:
Write an ARM assembly program that turns on a single LED by writing the appropriate value to the LED memory address.

- **Objective:** Familiarize students with memory writing and binary values.
- **Difficulty:** Very low.

Exercise II - Blink a single LED

Problem Description:
Write an ARM assembly program that continuously blinks a single LED using a simple delay loop.

- **Objective**: Introduce loops for delays and handling LED states.
- **Difficulty**: Low.

Exercise III - Read a Button without Affecting LEDs

Problem Description:
Write an ARM assembly program that reads the state of a push button and stores it in a register, without modifying the LEDs.

- **Objective**: Teach students how to read from memory without changing output values.
- **Difficulty**: Low.

Exercise 4 – Binary Counter on LEDs

Problem Description:

Write an ARM assembly program that **displays a binary counter on the LEDs**, which **increments from 0 to 255** (0x00 to 0xFF) in a loop.
Each step should **have a visible delay** so the counting is observable.

Objectives:

- Use **loops** to **increment a value** and **display it on the LEDs**.
- Use **subroutines (BL)**.
- Use **PUSH/POP** inside the subroutine.
- Practice **delay loops**.

Exercise 5 – Control LEDs with a Push Button

Problem Description:

Write an ARM assembly program that **reads the state of a push button** and **turns on all LEDs while the button is pressed**.

When the button is **released**, the **LEDs should turn off**.

- **LEDs Base Address:** 0xFF200000
- **Push Button Base Address:** 0xFF200040
  (Reads 1 when pressed, 0 when not pressed)

Objectives:

- Practice **reading GPIO input (button)** and **writing GPIO output (LEDs)**.
- Use **conditional branches** (CMP, BEQ, BNE).
- Use **subroutines with BL**.
- Use **PUSH/POP** as part of good practice for the subroutine.

Exercise 6 – Toggle LEDs with Push Button

Problem Description:

Write an ARM assembly program that **toggles the state of all LEDs each time Push Button 0 is pressed**.

The program should **detect the button press and release** to avoid multiple toggles from a single press.

- **LEDs Base Address:** 0xFF200000
- **Push Buttons Base Address:** 0xFF200050

Key Concepts Practiced:

- **Polling input** (waiting for button press/release).
- **EOR for toggling**.
- **Subroutine (BL) with PUSH/POP**.

Expected Behavior:

- Press Button 0 → LEDs toggle (ON/OFF).
- Each press-release cycle changes the LED state.
- Runs indefinitely.

Exercise 8.0 – Display the Number ‘7’ in the Seven-Segment Display

Problem description:

Write an ARM assembly program that directly displays the number **7** on a seven-segment display. The program should:

- Load the correct **seven-segment display pattern** for the number 7.
- Store the pattern in the **seven-segment display register**.
- Use a **lookup table** to retrieve the correct segment pattern.
- Keep the program simple, without requiring user input or button presses.

Segment Patterns for the Seven-Segment Display (gfedcba) [common cathode, 7-segment]:

Digit	Segments (gfedcba)	Hex Value
0	0b00111111	0x3F
1	0b00000110	0x06
2	0b01011011	0x5B
3	0b01001111	0x4F
4	0b01100110	0x66
5	0b01101101	0x6D
6	0b01111101	0x7D
7	0b00000111	0x07
8	0b01111111	0x7F
9	0b01101111	0x6F
A	0b01110111	0x77
b	0b01111100	0x7C
C	0b00111001	0x39
d	0b01011110	0x5E
E	0b01111001	0x79
F	0b01110001	0x71

Seven-Segment Display Base Address: 0xFF200020

Exercise 7 – Display the Number of Button Presses Using LEDs

Problem Description:

Write an ARM assembly program that **counts the number of times Push Button 0 is pressed** and **displays the count in binary using the LEDs**.
The counter should increase **each time the button is <u>pressed and released</u>**.
If the counter exceeds **255 (0xFF)**, it should **wrap around to 0**.

- **LEDs Base Address:** 0xFF200000
- **Push Buttons Base Address:** 0xFF200040

Exercise 8 – Display the Number of Button Presses Using the Seven-Segment Display

Problem Description:

Write an ARM assembly program that **counts the number of times Push Button 0 is pressed** and **displays the count as a hexadecimal digit (0 to F) on the rightmost digit of the seven-segment display**.

Each time **the button is pressed and released**, the counter **should increase by 1**.
If the counter **exceeds 15 (0xF)**, it should **wrap around to 0**.
To **display the numbers correctly**, you must use a **lookup table** with the segment patterns for the hexadecimal digits **(0 to F)**.

Devices and Addresses:

- Push Buttons Base Address: 0xFF200050
- Seven-Segment Display Base Address: 0xFF200020
- Each digit of the display is controlled independently, but you only need to use the rightmost digit.

You **must create a lookup table in memory** with these hex values, and **use the counter as an index** to get the correct pattern.

Objective:

- Use **PUSH/POP** to **protect registers** in subroutines.
- Use **BL** to **call subroutines** for updating the counter and displaying the number.
- Practice **arrays and indexing** using **lookup tables** in assembly.
- Interact with **Push Buttons and Seven-Segment Display**.

Key Assembly Concepts Used:

✅ GPIO interaction with 7-segment display
✅ Lookup table for segment patterns
✅ Indexing into a memory array
✅ PUSH/POP for subroutine safety
✅ Subroutine calls with BL

Expected Behavior:

- **Each press and release of Push Button 0 increases the counter by 1**.
- **The rightmost digit of the seven-segment display shows the number (0 to F)**.
- **After F (15), the counter wraps back to 0**.

Hint:

- **Create a lookup table** using .byte to **store the segment patterns**.
- **Access the table** using the counter as an index.